

BSP k-Means

Sebastian Künzel
sebastian.kuenzel@visus.uni-stuttgart.de
University of Stuttgart
Stuttgart, Germany

Daniel Weiskopf
daniel.weiskopf@visus.uni-stuttgart.de
University of Stuttgart
Stuttgart, Germany

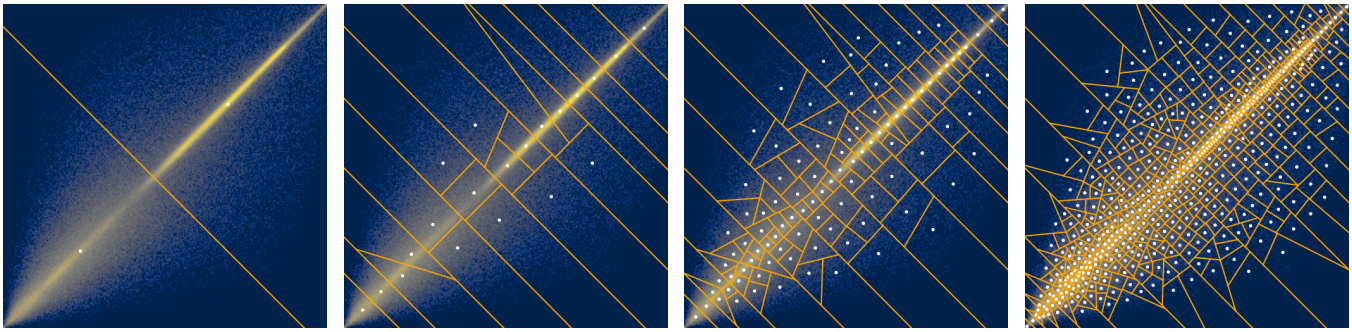
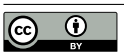


Figure 1: Space partitioning (orange lines) and cluster centers (white dots) computed using our k-Means clustering algorithm (without post-processing). Datapoints in each cell are assigned to the cell's corresponding cluster center. The background shows a log-scaled histogram of the 2D dataset (n = 524,800). From left to right: varying numbers of clusters k = 2, 20, 100, 500.

Abstract

We present a parallel k-Means clustering algorithm specifically designed to address large datasets and cluster counts. Our approach, *BSP* (binary space partitioning) *k-Means*, recursively partitions the input space optimally across the first principal component of the current subset of the data. From a single space partitioning, clusterings with arbitrary k can be found by a greedy tree search. Combining a reduction onto the first principal component and optimal binary clustering of 1D data is highly effective at reducing clustering costs. We attribute this to the close relationship between directional variance and the within-cluster sum of squares. We analyze our approach theoretically and, under an idealized population model, prove that the runtime is in $O(n \cdot \max(d, \ln n) \cdot \ln n)$ for a broad class of input distributions with high probability. We demonstrate that our algorithm consistently outperforms *k-Means++* and performs similarly or better than other sophisticated algorithms in an evaluation with nine datasets and up to six cluster sizes. Our algorithm is parallelizable using common database operations such as Scan, Reduce, and Sort. We show that with our CUDA implementation, a significant speedup can be achieved. Additionally, our algorithm generates centers and datapoint assignments alike, removing the expensive cluster assignment step after initial seeding. Furthermore, our algorithm approximates k-Means accurately so that any additional use of Lloyd's algorithm is optional and yields only incremental improvement.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780248

CCS Concepts

• **Computing methodologies** → **Cluster analysis**; **Parallel algorithms**.

Keywords

k-Means, Clustering, Parallel Computing

ACM Reference Format:
Sebastian Künzel and Daniel Weiskopf. 2026. BSP k-Means. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780248

Resource Availability:
The source code of our implementation is publicly available at https://doi.org/10.18419/DARUS-5575.

1 Introduction

Despite its age, k-Means is still a highly popular clustering algorithm and widely used in data analysis [7, 15]. With ever-growing dataset sizes, practical implementations often become limited by their poor computational complexity. In the last two decades, many significant improvements to k-Means clustering have been published. Some authors focus on improving clustering cost as defined by Arthur and Vassilvitskii [4], both in theory and in practice. Others focus on improving parallelism, computational complexity, or machine runtime in practice [4, 7, 15, 18].

We found that there is no algorithm that allows computing high-quality k-Means clusterings for large datasets with a large cluster count k. Almost all state-of-the-art algorithms scale linearly with k and/or are dependent on subsequent local optimizations using Lloyd's algorithm to generate competitive clusterings. Additionally, many state-of-the-art algorithms cannot be easily parallelized and

follow-up algorithms intended to address this issue required significant additional work and redesign, and suffer from other drawbacks such as overall increased computation.

In this work, we present a k-Means clustering algorithm specifically designed for large datasets with large cluster counts. We call our algorithm *BSP k-Means* because it recursively partitions the dataset in a binary fashion along the first principal component of the respective subset of the data, reminiscent of binary space partitioning (BSP). We emphasize practical usefulness, as demonstrated by an in-depth cost and runtime evaluation using real and synthetic datasets, comparisons to state-of-the-art algorithms, and many clustering scenarios. We focus on effectiveness, low memory footprint, and acceleration using a parallel GPU implementation. Finally, our theoretical analysis provides bounds on the time and space complexity of our approach. Our main contributions are:

(1) *BSP k-Means*, a novel, efficient, and effective algorithm for k-Means clustering.
(2) An extensive comparative evaluation of six algorithms, including state-of-the-art and state-of-the-industry algorithms, with nine datasets and up to six cluster counts.
(3) A parallel GPU implementation of *BSP k-Means*.
(4) Theoretical runtime bounds in $O(n \cdot \max(d, \ln n) \cdot \ln n)$ under an idealized population model, for log-concave input distributions with n datapoints in d-dimensional space.

2 Background and Related Work

Given n datapoints in $\mathbb{R}^d$ from a set S, optimal k-Means clustering is a k-partitioning of S in non-empty subsets $S_1, S_2, \ldots, S_k$ that minimizes the within-cluster sum of squares (WCSS) [32]:

$$WCSS = \sum_{i=1}^{k} \sum_{x \in S_i} ||x - \mu_i||^2.$$

It has been shown that finding an optimal solution for the general and even planar k-Means problem is NP-hard [3, 16, 26]. A globally optimal solution is impractical in almost any application. This theoretical limitation, combined with the high popularity of k-Means clustering in data analysis [7], led to a plethora of polynomial-time algorithms that aim to approximate optimal k-Means clustering.

An important category of k-Means approximation algorithms are constant-factor approximations. Algorithms of this category approximate the optimal solution within a small constant multiple of it. Currently, the best approximation ratio (to the best of our knowledge) is 5.912 for an algorithm by Cohen-Addad et al. [14], improving upon an approximation ratio of 6.129 for an algorithm by Grandoni et al. [17] and further upon Ahmadian et al. [2]. While these polynomial-time algorithms have impressive proven bounds on their approximation quality, they suffer from large constant exponents governing their computational complexity, rendering them impractical. These algorithms are mainly of theoretical interest.

One of the first practical approximation algorithms is based on results by Lloyd [24] and is still widely used due to its simplicity and trivial parallelization [4]. After drawing k initial seeds from S in a uniformly random fashion, Lloyd's algorithm is applied until the clustering cost sequence converges to a local minimum. Lloyd's algorithm repeatedly cycles two subroutines [4]. First, each datapoint is assigned to the closest center candidate. If linear search is used, computational complexity is in $O(ndk)$ and the subroutine is trivially parallelizable with $O(nd)$ memory needs. Second, new center candidates are formed by computing the centers of datapoints with equal assignments. The computational complexity of this second subroutine is in $O(nd)$ and negligible compared to the first subroutine. Theoretical results show that random initialization followed by Lloyd's algorithm can lead to arbitrarily poor clusterings with respect to clustering cost. This can be confirmed empirically by comparing with other approaches [4]. As Lloyd's algorithm always improves clustering cost until a local minimum is reached, subsequent research focused on improved seeding [4, 22], followed by a small number of local optimization steps using Lloyd's algorithm. As long as the seeding algorithm is in $\Omega(ndk)$, using a small number of steps in Lloyd's algorithm is not a bottleneck.

A popular seeding algorithm is *k-Means++* [4]. It chooses initial centers from a squared-distance distribution, improving upon random seeding in both theory and practice. Arthur and Vassilvitskii [4] showed that initial centers chosen with *k-Means++* are within an $O(\log k)$ ratio from the optimal solution and that *k-Means++* seeding also significantly reduces clustering cost in practice. The computational complexity is in $O(ndk)$ in general and thus similar to Lloyd's algorithm. In practice, clustering is often performed by an initial *k-Means++* seeding and a small number of local optimization iterations using Lloyd's algorithm. Both random initialization and *k-Means++* initialization are implemented in well-known ML libraries such as scikit-learn [31].

A drawback of *k-Means++* is the inherent sequential selection of initial centers [7], preventing parallelization of the initialization and introducing a bottleneck when clustering large datasets. Bahmani et al. [7] designed an algorithm inspired by *k-Means++* that utilizes parallelism to speed up clustering. However, its computational complexity is higher than *k-Means++*, and a post-processing step further limits performance for large k. Hess et al. [18] improved upon Bahmani et al. by introducing a distributed seeding algorithm with a constant number of computation rounds.

For big data, coresets can be used to summarize data and reduce the problem size. An evaluation of existing k-Means coresets is given by Schwiegelshohn and Sheikh-Omar [34]. The coreset sizes in their evaluation are set to $200k$, retaining a linear dependency on k, even when computing k-Means on a coreset. This highlights the necessity for new methods that are independent of k or at most logarithmically dependent on it.

Others focused on improving the accuracy of *k-Means++*, rather than its parallel performance. Lattanzi and Sohler [22] included additional rounds of local search after the initial *k-Means++* seeding to achieve a constant approximation ratio to the optimal solution. Choo et al. [13] showed that a constant number of local search rounds yields a similar approximation guarantee. Beretta et al. [8] further refined the analysis of the local search, improving bounds and showing that a more elaborate search can further decrease the cost of the initial centers. However, the issues regarding parallelization are similar to those of *k-Means++*.

Bachem et al. [5] could speed up *k-Means++* seeding by sampling center candidates using a Markov chain. However, their approach is only suitable for a small number of clusters [15], and it loses its benefits if the datapoints are to be assigned to their closest center. Cohen-Addad et al. [15] proposed a multi-tree embedding and

rejection sampling that achieves log-linear runtime with data represented by machine numbers. They compute clustering cost values by assigning each datapoint to its closest center using linear search. Therefore, only cluster seeding can be computed in log-linear time, whereas the clustering including datapoint assignment and cost is still in $O(ndk)$. We found that their reference implementation includes an interface for native datapoint assignments computed during seeding. However, these assignments are not used in their reference implementation and not mentioned elsewhere. Therefore, we decided to ignore this option in our evaluation. Our tests showed that the native datapoint assignments are not competitive with any evaluated algorithm by a large margin.

Despite the many advances in k-Means clustering, there is no algorithm for clustering large data sets, especially for large cluster numbers k. Furthermore, many of the existing algorithms are not designed for parallel execution. We address these issues by introducing *BSP k-Means*, and demonstrate its benefits by a comparative evaluation with the relevant representatives of the state of the art.

Finally, there are many tree-based and hierarchical approaches to approximate a k-Means clustering solution. Our approach improves on many of those, including principal component trees and hierarchical k-Means. An overview of tree data structures for k-Means is given by Tavallali et al. [35]. In the next section, we point out similarities and key differences compared to our approach.

3 Method

In this section, we first describe our approach: *BSP k-Means*. Then, we discuss aspects of its memory and computational complexity. Empirical evaluation follows later in Section 4.

3.1 Recursive Computation by BSP k-Means

BSP k-Means recursively subdivides the input space, as in hierarchical k-Means, using binary space partitioning to create a BSP tree. Every node n is either subdivided into two children or terminated as a leaf. The mean of all datapoints assigned to n is an optimal center given the assignment. Similar to principal component trees, we subdivide each node perpendicular to the first principal component of the datapoints assigned to the node. However, instead of computing a full decomposition and splitting at the mean or median, we approximate only the first principal component using a minimization algorithm similar to a related algorithm by Kwak for L1-PCA [20] that iterates over the data for a small, constant number of rounds. We then project the datapoints using the first principal component and choose a splitting point using optimal 1D two-clustering of the projected data, similar to Otsu's method [29].

Otsu's method selects a split point t that maximizes the between-class variance of the projected 1D data:

$$\sigma_b^2(t) = w_0(t)w_1(t)\left(\mu_1(t) - \mu_0(t)\right)^2,$$

where w_0, w_1 are the class weights (proportions of data left and right of t), and μ_0, μ_1 are the corresponding class means [29].

We continue subdividing nodes until either the desired number of clusters is obtained or every node has a variance of zero. All nodes that are not yet subdivided and that are not leaf nodes form a frontier. For every node n in the frontier, $\Delta cost$ is found by expanding the node n, computing the costs of its children, and having $\Delta cost = cost_n - (cost_{lchild} + cost_{rchild})$. This is the cost decrease manifesting when n is expanded, with $cost_n$ as the node's cost and $cost_{lchild}$ and $cost_{rchild}$ as the costs of the respective children. Each node to be subdivided is picked from the frontier so that $\Delta cost$ is largest among all nodes in the frontier. This constitutes a greedy minimization of the clustering cost. Each subdivision removes one node from the frontier and adds two new nodes, the subdivided node's children.

At some point, the frontier contains k nodes. The centers are then given as the node means, the WCSS is the sum of individual node costs, and the datapoint assignment is the union of all nodes' datapoint assignments. If the removal order of nodes from the frontier is tracked, all clusterings up to k are accessible from the nodes that are currently in the frontier or were previously in it. As consecutive frontiers differ by exactly two nodes, all clusterings up to k can be stored in $O(kd)$ memory. Similarly, the new WCSS after subdivision can be computed by simply subtracting $\Delta cost$ from the previous WCSS. As WCSS costs for each value of k become available during subdivision, the elbow method is immediately applicable [36]. We demonstrate the elbow method in Appendix B.

In the context of this work, subdivision is set to stop when all nodes have a variance of zero. This constructs a full BSP tree of the dataset and generates all possible clusterings up to $k_{\max}$, where $k_{\max}$ is given by the number of unique datapoints in the original dataset. Any further increase in k will not improve cost as WCSS is already zero at $k_{\max}$. The pseudocode is provided in Algorithm 1.

Algorithm 1 BSP k-Means Pseudocode

Input: Dataset $D \subset \mathbb{R}^d$, center count k
Output: Map of nodes M

Function `subdivide`(n)**:**
  Compute mean for datapoints D_n
  Compute cost c for datapoints D_n
  if $c = 0$ **then**
    return
  end if
  Compute principal component (PC) for D_n
  Project D_n onto PC
  Sort projections of D_n
  Apply sort order to data
  Find optimal split index i_s that maximizes σ_b^2
  return (node(n_{start}, i_s), node(i_s, n_{end}))
End Function

Initialize M as map of $\Delta cost$ and nodes
Add $\Delta cost_{root}$ and the root node(0, size(D)) to M
while size(M) $< k$ **do**
  Pop the node n with largest $\Delta cost$ from M
  Subdivide the node n using `subdivide`(n)
  Add the child nodes with respective $\Delta cost$ to M
end while
return M

Let us explain the reasoning behind our approach. Splitting the data perpendicular to the first principal component ensures that

centers are spread in the direction of largest variance, reducing clustering cost most effectively. Additionally, repeatedly splitting the data along the first principal component increases isotropy in the subsets, counteracting high-dimensional deterioration. For high dimensionality, datapoints would otherwise tend to populate the fringes of node volumes that, combined with the exponentially increasing neighbor count, would cause problems for many tree- and especially grid-based techniques. By using optimal 1D two-clustering of the data to determine a splitting point, we avoid splitting areas of high density. This minimizes data density at the node boundaries and therefore minimizes misplacement of a datapoint. Our design intentions are confirmed by our experiments, showing increasing isotropy and splitting in areas of low density. The experimental results are given and explained in Section 4.4. Furthermore, repeated quasi-optimal two-clustering also steers toward a good overall k clustering. This is shown by our clustering cost experiments in Section 4.1. Finally, we use a greedy search to expand our clustering solution in the area that improves cost the most. We carefully construct this constructive interference of heuristics to create a BSP tree that is well-suited for k-Means clustering.

3.2 Parallel BSP k-Means

Clustering can greatly benefit from distributed and parallel computing. However, for many algorithms that approximate k-Means, a parallel mapping is not trivial to find and cannot be expected. In the case of *k-Means++*, a parallel mapping with similar performance was proposed by Bahmani et al. more than five years after the sequential implementation, and the parallelization required that almost all aspects of the initial algorithm had to be changed [7]. Many of the algorithms discussed in Section 2 have no parallel counterparts to date. In summary, parallelization tends to be hard for many clustering algorithms.

Therefore, we consider it a major advantage of our approach that it readily lends itself to a parallel GPU implementation. More importantly, parallelization achieves a significant speedup of one order of magnitude (see Table 1) and relies entirely on parallel operations and primitive database operations such as *Reduce*, *Scan*, and *Sort*. We implemented our prototype in CUDA using device-wide primitive operations from the CUDA Core Compute Libraries [12]. *BSP k-Means* as described in Section 3.1 is changed in two significant aspects. First, the tree is generated on the GPU layer by layer in parallel up to a predefined depth. Second, the frontier is propagated through the tree on the CPU to yield k clusters. Essentially, the tree generation is decoupled from the frontier propagation. Since the frontier propagation is significantly cheaper than the tree generation, this change establishes an efficient hybrid algorithm for parallel k-Means clustering.

3.3 Post Processing

Once the BSP tree is generated and k clusters are determined, the existing tree structure can be utilized to further improve clustering cost with little overhead. The subtree extending from the root to the k nodes of the final frontier is used to approximate a neighborhood for each node in the final frontier. We do this by going through the parents of a node upward to the root, assigning nodes from opposing branches until a logarithmic (in n) number of approximate neighbors is found. Knowing an approximate local neighborhood of each node, we can run a limited Lloyd's optimization that assigns datapoints within the approximate neighborhood of the current cluster. Limiting the search space to the approximate local neighborhood allows us to avoid the bottleneck of a linear search, utilizing the hierarchy and proximity of centers already encoded in the BSP tree.

3.4 Analysis

We analyze the time and space complexity of *BSP k-Means*.

Space Complexity. The memory required per node is at most in $O(d)$ if a center is stored for each node. The number of nodes in the terminating frontier is bound by $k_{\max} < n$, given n datapoints. Therefore, the total number of nodes is in $O(n)$ and the total space complexity is in $O(nd)$. This also includes the dataset itself and a constant amount of auxiliary storage, such as datapoint assignments, cost values, and all possible clusterings up to k.

BSP Tree Generation. Understanding computational complexity is more involved. We start by considering the complexity of processing a single node. The center, cost, principal component (using a minimization algorithm), and projection of a node are computed with time complexity in $O(n_{\text{node}}d)$. Sorting of the projections is computed with time complexity in $O(n_{\text{node}} \ln n_{\text{node}})$ using merge sort. Finding the split index is in $O(n_{\text{node}})$ and thus absorbed by the complexity of sorting. Subdividing a node does not produce new datapoints but may terminate as a leaf. Hence, at any layer of depth D in the BSP tree, there are at most n datapoints. The overall time complexity of processing a layer is therefore in $O(n \cdot \max(d, \ln n))$. The BSP tree generation is bound by the depth of the BSP tree. Let $D_{\max}$ be the maximal depth of the tree. Then, we need to derive $D_{\max}$ to understand the complexity of the tree construction.

Let us consider log-concave distributions as input to arrive at $D_{\max}$. Log-concavity is a standard assumption in theoretical analyses due to its generality and desirable properties [11, 33]. Log-concave distributions include many highly relevant distribution types, including the normal distribution and the uniform distribution. Given a log-concave input distribution, for every confidence level $\delta \in (0, 1)$, with probability at least $1 - \delta$, the sample-labeled population tree depth satisfies:

$$D_{\max} \leq \frac{\ln(n/m_{\min}(\delta))}{\ln(1/(1-\frac{c}{2}))} + m_{\min}(\delta)\,, \quad m_{\min}(\delta) = \frac{2}{c^2}\ln\left(\frac{4n}{\delta}\right).$$

with c as a universal constant. Appendix A provides a proof for these bounds. An extension of this proof extends this result to mixtures of log-concave distributions, as long as the component weights are globally bound by a constant; see Appendix A.5.

Therefore, the overall computational complexity of the sample-labeled population tree generation is in

$$O(n \cdot \max(d, \ln n) \cdot \ln n).$$

The sample-labeled population tree serves as the population-level limit of the empirical Otsu tree and captures its large-sample behavior. The complexity bound holds for all log-concave input distributions with high probability and—most importantly—it is independent of k. Our theoretical results are supported by our experiments discussed in Section 4.4. Furthermore, our experiments suggest

that favorable runtime and depth bounds extend beyond the log-concavity assumption of the proof.

Additional analysis reveals that both best-case and worst-case runtime scenarios, specifically runtime in $O(nd)$ and runtime in $O(ndk)$, seem to occur with pathological data only. This includes data collapsed into copies of a single point or data spaced at exponentially increasing distances.

Frontier Propagation. The frontier is propagated from the root, expanding one node at a time. Maintaining a map of cost deltas and expanding $k-1$ nodes to yield a frontier of size k is of complexity in $O(k \ln k)$. Compared to tree generation, frontier propagation is negligible from a computational perspective.

Post Processing. We introduced a post-processing local optimization that uses the BSP tree to find $O(\ln n)$ close neighbors that are promising candidates for data reassignment to better clusters. Finding this set of neighbors for each cluster center is computed with complexity in $O(k \cdot (\ln n)^2)$, as the tree depth is in $O(\ln n)$ and the number of neighbors searched is in $O(\ln n)$. Our post-processing, operating with a reduced set of neighboring clusters to compare against, then computes in $O(nd \cdot \ln n)$. Thus, the overall complexity of the post processing is comparable to the BSP tree generation but with significantly lower constant factors in practice.

4 Experiments

In our evaluation, we compare *BSP k-Means*, short BSP, with six algorithms from the literature on nine datasets with six cluster counts for six of the datasets and three cluster counts for three of the datasets. We provide our reference implementation generating the reported results as open source [21].

We chose six datasets for extensive evaluation and three datasets for auxiliary evaluation. We preferred frequently used datasets from previous literature to allow cross-validation and comparison. We could not always find the exact datasets used in previous literature. We removed all non-feature columns, such as indices and class labels. In the order of listing: *Protein* KDD Cup 2004: Protein homology data [30], used previously by Cohen-Addad et al. [15]; *Song*, audio features from western music [9], used previously by Cohen-Addad et al. [15] and Bachem et al. [5]; *Census*, US census data [28], used previously by Cohen-Addad et al. [15] and Ackermann et al. [1]; *Cover*, geological data [10], used previously by Ackermann et al. [1] (here, we removed the last 40,070 rows to remove bad data); *Higgs*, particle physics data [37], used previously by Hess et al. [18]; *Gauss*, a synthetic Gaussian mixture dataset generated using sample code from Hess et al. [18]. *Gauss* is composed of 50 Gaussian distributions in $\mathbb{R}^{15}$ and 10^7 sample points total, including 20 invalid rows (-10^{66}) to test the evaluated algorithms for robustness.

We used two datasets not found in previous literature. *High* is a synthetic Gaussian mixture of 100 Gaussian distributions in $\mathbb{R}^{512}$ with 99,950 datapoints in total. With *High*, we investigated performance in higher dimensions, observing a turning point in cluster cost at $k = 100$. Given the construction of this dataset, at $k = 100$, every Gaussian distribution in the mixture should be sampled with one cluster center for low cost. One can expect marginal cost decreases for $k > 100$ and huge cost increases for $k < 100$. *High* was generated similar to *Gauss*. *Image* is a 2D dataset derived from horizontally adjacent intensity values in a scaled grayscale image of a lake landscape. We used *Image* to investigate cluster center distributions generated by the algorithms. Finally, we use the *Mnist* dataset, a widely used benchmark known as the "MNIST Database of Handwritten Digits" [23]. We chose $k \in \{50, 100, 500, 1000, 5000, 10000\}$ for our evaluation based on common values of k used in previous evaluations, such as those listed above and in Section 2. *High*, *Image*, and *Mnist* were evaluated using a reduced set of cluster counts $k \in \{50, 100, 500\}$, as there are no further insights gained from continued evaluation.

We compare *BSP k-Means* against two fast state-of-the-art algorithms: *Reject* and *Fast* by Cohen-Addad et al. [15], with lower computational complexity compared to most other approaches and proven bounds on accuracy. With this comparison, we aim to show that our approach yields lower cost than a state-of-the-art algorithm with similar computational complexity. Additionally, we compare *BSP k-Means* against *Local*, an accurate state-of-the-art algorithm with uncompetitive computational complexity, by Lattanzi and Sohler [22]. *Local* has strong theoretical bounds [8]. This comparison shows that our approach yields similar cost as an accurate state-of-the-art algorithm with worse computational complexity. We compare *BSP k-Means* against *SOCCER* with $\epsilon = 0.01$, an algorithm with strong bounds by Hess et al. [18]. With this comparison, we aim to show that our approach has superior parallel performance compared to a state-of-the-art parallel algorithm.

Moreover, we compare *BSP k-Means* against *k-Means++*, a widely used state-of-the-industry approach by Arthur and Vassilvitskii [4]. *k-Means++* features good theoretical bounds, strong practical results, and a simple implementation. With this comparison, we demonstrate that our approach consistently outperforms a state-of-the-industry algorithm. Finally, we compare *BSP k-Means* against *Rand*, a baseline of uniform random sampling followed by assigning each datapoint to the closest center. Any approach, including ours, should yield lower cost than the baseline algorithm.

We implemented all algorithms according to the literature, except for *Reject*, *Fast*, and *SOCCER*. We included *Reject* and *Fast* using unmodified reference code by the author Norouzi-Fard from a repository provided by the University of Hamburg. An interface for their implementation was created. We used the standalone code provided in a repository by Hess et al. [18] for our evaluation of *SOCCER*. *SOCCER* did not provide results for all of our test cases; missing values are indicated in the tables below. All algorithms were evaluated on a desktop computer (Windows 11, Intel i7-13700T, 64 GB RAM, NVIDIA RTX4090). All algorithms used a single CPU thread, except for *SOCCER* and to some extent *Fast* and *Reject*.

4.1 Cost Evaluation

For each algorithm, we provide cost values for the initial clustering solution without further local optimization using Lloyd's algorithm. Detailed measurement results are listed in Table 2. To compute cost values, each datapoint needs to be assigned to a center. Ideally, each datapoint is assigned to the closest center, minimizing WCSS given a set of centers. Both *Local* and *k-Means++* keep track of the closest centers when implemented properly. *Rand* requires an assignment run, akin to the first subroutine of Lloyd's algorithm [24],

to compute a cost value. Both *Reject* and *Fast* use a closest-center assignment to compute cost values in the given reference implementation. Our approach *BSP k-Means* outputs cost values and approximate datapoint assignments without using an additional exact assignment routine in $O(ndk)$. For *BSP k-Means*, we report cost without and with post processing (C_p, see Section 3.3) using the BSP tree. Cost values in Table 2 are scaled by a factor s to fit the table; s is given in a separate column.

From Table 2, it is evident that our *BSP k-Means* consistently reports lower cost than competing algorithms. The only exception is *SOCCER*, which is on par with *BSP k-Means*. *BSP k-Means* substantially outperforms *k-Means++*, often by more than fifty percent. Our evaluation confirms that *Local* improves on *k-Means++* in practice but falls short of our approach. *Reject* and *Fast* are consistently outperformed by *Local* and *BSP k-Means* and perform mostly similar to *k-Means++*. Our results for both the *Song* and *Census* datasets match the evaluation of Cohen-Addad et al. [15]. For the *Higgs* dataset, *Rand* outperforms *k-Means++*, *Reject*, and *Fast* for some values of k by a small margin but is outperformed by *Local* and our approach *BSP k-Means* as well as *SOCCER*.

Most outstanding are results for the *Gauss* and *High* datasets. Both datasets are Gaussian mixtures—with 50 distributions for *Gauss* and 100 distributions for *High*. For k smaller than the number of distributions, a high cost is to be expected, as not all distributions are covered by at least one center. For k equal to or larger than the number of distributions, low cost with a gradual decrease toward higher values of k is expected, as ideally all distributions should be covered by at least one center. Therefore, for *Gauss*, low cost with gradual decrease is expected, and this is indeed the case for *BSP k-Means*, *Local*, and *k-Means++*. In contrast, *Reject*, *Fast*, and *Rand* fail to adequately distribute centers among the distributions and therefore yield clusterings orders of magnitude worse compared to other approaches. For very large k, there is some improvement, as the probability of covering all distributions at least once increases. *SOCCER* completely fails on *Gauss*, potentially getting confused by the 20 outliers included. *High* is even more interesting. First, *High* is of significantly higher dimensionality than the other datasets. Second, at $k = 100$, a change in cost decrease is expected. At $k = 50$, the dataset is underclustered, yielding high cost for all algorithms. Still, our *BSP k-Means* yields the lowest cost for $k = 50$ by a substantial margin. A substantial decrease in cost for $k \geq 100$ is observed for *BSP k-Means*, *Local*, and *k-Means++*. *Reject*, *Fast*, and *Rand* fail again to adequately distribute centers among the distributions and yield very poor results. *SOCCER* correctly distributes centers for $k = 100$ and has costs similar to our approach.

Lloyd's algorithm can be applied to any set of centers to let cost values converge to a local minimum. However, this algorithm is rather expensive and frequently converges to poor local minima [4]. Our evaluation found that after ten iterations of Lloyd's algorithm, cost values are close enough to convergence for all datasets tested. Let us consider cost values obtained from our approach *BSP k-Means* again. They are computed using datapoint assignments generated by *BSP k-Means* without any additional data assignment routine or Lloyd's algorithm. Cost values C_p are, on average, 3% higher than those after ten subsequent optimization rounds using Lloyd's algorithm, with a standard deviation of 3.8%. The quality of the clustering solutions after ten optimization rounds using Lloyd's algorithm is similar to before, with smaller margins. Our approach *BSP k-Means* and *SOCCER* yield the lowest cost, with competing algorithms following in the established order. We conclude that our approach *BSP k-Means* makes any subsequent local optimization using Lloyd's algorithm optional, incremental, and negligible. Thus, our algorithm is a clustering algorithm rather than a seeding algorithm for Lloyd's algorithm.

4.2 Memory Usage Evaluation

For each algorithm, we measured peak memory usage during initial clustering. Peak memory usage measured during algorithm evaluation is shown in Table 3. Memory usage for *Rand* is limited to the size of the dataset and the number of sampled center points. Both *k-Means++* and *Local* only marginally increase memory usage. *BSP k-Means* uses slightly more than three times the memory used by *Rand* in the worst evaluation sample and much less in most other evaluation samples. *Reject* and *Fast* suffer from excessive memory usage. Since we used a reference implementation provided by the authors, it remains an open question whether the excessive memory usage is caused by a resource-intensive implementation or if it is inherent to the algorithm. We do not exclude that improvements toward lower memory usage are possible for both algorithms. *SOCCER* uses considerably more memory than our approach but still less than *Reject* and *Fast*.

4.3 Runtime Evaluation

Machine runtime was measured until initial clustering was completed. Measurements are presented in Table 1. *Reject* and *Fast* can output datapoint assignments required for a clustering solution in the presented time, but the authors of these previous works decided not to use or mention this output in their work (although present in their reference code). Giving the benefit of the doubt, we recorded the time elapsed until seeding was completed for *Reject* and *Fast*, ignoring the subsequent computation used to determine the cost as implemented by the authors. It is important to note that without this additional optimal datapoint assignment, the clusterings generated by *Reject* and *Fast* are significantly worse than the reports in Table 2. *SOCCER* uses all available CPU threads, *BSP GPU* is executed on the graphics card.

Our evaluation confirms theoretical results on algorithm complexity. While *BSP k-Means*, *Reject*, and *Fast* have complexities independent of k or logarithmically dependent on k, runtime increases proportionally to k for *Local*, *SOCCER*, *k-Means++*, and *Rand*. Our approach *BSP k-Means* was set up to construct a full BSP tree and not stop early once the desired k is reached. *BSP k-Means* outperforms *Reject* and *Fast* for all datasets except *Gauss* and *High*. Compared to *Local*, *SOCCER*, *k-Means++*, and *Rand*, there exist break-even points between $k = 500$ and $k = 5000$. It is evident that *BSP k-Means* outperforms *Local*, *k-Means++*, and *Rand* by orders of magnitude for large datasets and large k. *SOCCER* is exceptionally fast for small k and exceptionally slow for large k. *BSP k-Means* using CUDA acceleration is significantly faster than the CPU counterpart and reports break-even points between $k = 100$ and $k = 500$.

Table 1: Machine runtime (in seconds) by algorithm, dataset (D), and cluster count k. A break-even point for our approach is clearly visible when compared against other algorithms that scale proportionally to k. *GPU* refers to the CUDA implementation of *BSP k-Means*, *SOCC* to *SOCCER*, and *kM++* to *k-Means++*.

D	k	**BSP**	**GPU**	Local	kM++	Rand	Fast	Reject	SOCC*
Protein	50	17.2	1.3	1.5	0.5	0.3	38.0	38.5	2.1
	100	17.1	1.3	2.9	1.0	0.6	38.0	38.3	5.2
	500	17.0	1.3	14.4	4.6	3.2	38.0	38.9	51.4
	1000	17.2	1.3	28.0	9.2	6.4	38.3	38.7	—
	5000	17.0	1.3	144.9	45.9	32.8	39.2	39.8	—
	10000	17.3	1.3	311.6	89.9	65.5	40.1	41.1	—
Song	50	36.9	6.7	3.3	1.0	0.7	85.7	87.2	3.2
	100	36.8	6.7	6.6	2.0	1.4	85.2	87.7	7.4
	500	36.8	6.7	29.8	10.0	7.0	86.3	88.2	121.1
	1000	36.8	6.7	62.4	20.6	14.1	90.9	88.1	256.7
	5000	37.0	6.7	313.7	103.8	72.5	87.4	89.9	—
	10000	37.1	6.7	650.6	201.2	145.3	88.8	89.9	—
Census	50	190.4	37.8	10.7	3.7	2.5	174.6	160.9	1.7
	100	190.6	37.8	23.9	7.2	5.0	177.3	162.4	4.5
	500	190.7	37.8	116.8	36.0	23.6	178.5	164.4	67.1
	1000	190.2	37.8	231.2	73.9	46.7	178.7	166.1	233.3
	5000	190.8	37.8	1147.4	374.8	239.3	181.8	171.9	5391.2
	10000	190.9	37.8	2271.0	727.7	486.0	182.8	186.5	—
Covertype	50	25.5	4.7	1.9	0.7	0.4	65.2	66.4	1.8
	100	26.0	4.7	4.4	1.4	0.9	64.7	66.6	4.0
	500	25.7	4.7	22.8	6.8	4.6	66.4	67.1	43.4
	1000	25.7	4.7	38.4	13.6	9.3	66.0	68.4	127.4
	5000	25.5	4.7	193.1	67.1	47.2	67.1	68.7	—
	10000	25.4	4.7	369.1	135.7	96.4	69.5	70.4	—
Higgs	50	487.2	37.3	22.6	7.4	5.0	610.7	604.6	1.7
	100	480.5	37.3	45.2	14.8	9.5	611.7	613.3	3.2
	500	478.8	37.3	228.7	71.7	49.5	620.3	615.4	59.0
	1000	480.4	37.3	703.8	144.0	98.5	630.4	623.6	235.1
	5000	490.0	37.3	3603.5	707.1	491.7	639.7	646.8	7703.4
	10000	481.2	37.3	7143.9	1403.5	999.7	638.1	677.7	26390.6
Gaussian	50	211.4	15.2	12.5	3.4	2.4	34.8	35.0	1.1
	100	213.1	15.2	37.7	7.2	4.6	34.9	35.1	2.2
	500	210.9	15.2	192.6	36.9	29.6	34.8	34.9	11.8
	1000	212.0	15.2	378.6	71.4	57.5	34.8	35.3	100.3
	5000	211.3	15.2	1095.6	361.6	274.4	35.4	47.5	1397.3
	10000	213.2	15.2	2170.9	698.9	465.2	35.0	91.6	6430.7
High	50	37.2	5.5	3.6	1.3	1.0	6.3	6.6	1.1
	100	36.8	5.5	7.7	2.5	2.0	6.3	6.4	2.7
	500	36.8	5.5	38.8	12.7	10.1	6.2	6.7	—
Image	50	1.1	0.1	0.2	0.1	0.0	4.8	5.2	0.8
	100	1.1	0.1	0.4	0.1	0.1	5.2	5.6	2.6
	500	1.1	0.1	1.5	0.5	0.4	5.9	6.3	22.3
Mnist	50	32.3	2.2	4.4	1.4	1.1	48.3	47.1	5.4
	100	32.6	2.2	9.0	2.8	2.2	47.4	47.2	14.3
	500	32.4	2.2	43.4	13.9	10.8	68.9	36.5	—

*Measurements for *SOCCER* represent a parallel algorithm using all available CPU threads.

4.4 BSP Tree Analysis

For each dataset, we computed a full BSP tree using our approach and measured average and maximum tree depth. We also report the logarithm of the dataset size. The results are shown in Table 4. There are no significant outliers for both average and maximum tree depth, and all maximum depth values are within $5 \cdot \ln n$.

Additionally, we computed the eigenvalues of the covariance matrix of each node's datapoints (principal components). Our experiments show that, on average, isotropy in a node's datapoints increases in deeper layers through subsequent splitting along the first principal component. The results are visualized in Figure 2. We exclude *Image*, as it is 2D, and *MNIST*, for which the BSP tree is too shallow, from the average. Furthermore, a density histogram of the node's data along the first principal component indicates that, on average, Otsu's method tends to split the data in areas of low density, as shown in Figure 3. The results support our assumption that our algorithm circumvents shortcomings of existing space partitioning algorithms in high dimensions. For high dimensionality, grid-based algorithms suffer from significant anisotropy and densely populated decision borders. Using our method, isotropy is increased and the split areas tend to be sparsely populated. This prevents ambiguous datapoint assignments on the fringes of the partitions.

5 Conclusion and Future Work

We have presented *BSP k-Means*, a novel and parallel k-Means clustering algorithm tailored toward large datasets with large cluster counts k. We have shown in an extensive evaluation that our approach outperforms or easily competes with other algorithms with regard to cost, speed, and memory usage.

First, *BSP k-Means* reports excellent cost values and consistently outperforms all competing algorithms except *SOCCER*, to which it is comparable. Second, *BSP k-Means* generates competitive clusterings without the need for additional local optimization or exhaustive datapoint assignment. Clustering cost improves only 3% on average over ten subsequent rounds of Lloyd's algorithm. This is an improvement compared to many competing algorithms and allows us to discard expensive local optimization. Third, *BSP k-Means* is memory-friendly, often using less than two times more memory than *k-Means++* across all evaluated datasets. Our approach can generate all k-clusterings and associated costs in a single run with memory usage in $O(nd)$. Fourth, *BSP k-Means* scales favorably with k. Our evaluation shows that computing all possible clusterings for $k < n$ results in average and maximum tree depths that are not much larger than $\ln n$. We have proven that this holds with high probability for log-concave data distributions and mixtures of log-concave distributions. We have reported break-even points from $k = 500$ to $k = 5000$ for our CPU implementation and $k = 100$ to $k = 500$ for our GPU implementation in comparison with *k-Means++* and other more expensive algorithms. Careful optimization may further lower the threshold. Our GPU implementation improves performance by one order of magnitude and can be realized by just using primitive database operations. We consider it a major advantage that we can arrive at a parallel mapping immediately.

In future work, the theoretical analysis of our algorithm could be extended further, mainly focusing on bounds on accuracy. Both our CPU and GPU implementation could be further improved, reducing overhead and memory consumption. Finally, our approach might be used to cluster extremely large datasets with high cluster counts, something that was previously hard to accomplish.

Ethical Use of Data and Informed Consent

No study involving human or animal subjects was performed for this work. The datasets used were either synthesized or acquired from the open repositories, without any ethical issues associated. Data processing with k-Means clustering concerns basic research with no direct ethical concerns.

Acknowledgments

This work was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy – EXC-2075 – 390740016. We thank Daniel Klötzl for his feedback. We used generative AI (ChatGPT) in the appendix of our work to help formalize concepts and improve the quality and clarity of our manuscript.

Table 2: Clustering cost scaled by a factor s and ordered by dataset (D) and cluster count k. Cost is to be minimized. Our *BSP k-Means* outperforms or performs equal to competing algorithms. *BSP k-Means* with post-processing is presented as C_p. We abbreviated *SOCCER* as *SOCC* and *k-Means++* as *kM++*.

D	k	s	**BSP**	C_p	Local	kM++	Rand	Fast	Reject	SOCC
Protein	50	10^8	4084	3883	5126	5935	20834	6068	6007	3980
	100	10^8	3472	3262	4278	4872	20056	4773	4813	3332
	500	10^8	2507	2339	2810	3155	15797	3100	3113	2287
	1000	10^8	2207	2046	2401	2630	5596	2645	2615	–
	5000	10^8	1603	1472	1670	1805	2089	1817	1809	–
	10000	10^8	1344	1238	1404	1517	1796	1522	1523	–
Song	50	10^9	1723	1642	2206	2390	2667	2435	2534	1674
	100	10^9	1558	1479	1959	2134	2401	2114	2219	1490
	500	10^9	1249	1186	1516	1632	1821	1673	1654	1165
	1000	10^9	1140	1082	1370	1464	1631	1485	1480	1049
	5000	10^8	8979	8558	10705	11485	12808	11550	11552	–
	10000	10^8	7863	7520	9386	10180	11634	10217	10262	–
Census	50	10^5	1349	1251	1947	2389	2417	2288	2423	1285
	100	10^4	10657	9855	14627	17246	18845	17433	17431	9879
	500	10^4	6526	5876	8425	9789	10510	9598	10127	5669
	1000	10^4	5421	4871	6742	7759	8476	7706	7874	4602
	5000	10^4	3645	3285	4287	4864	5208	4794	4841	2966
	10000	10^4	3103	2788	3579	4020	4325	3998	4032	–
Covertype	50	10^9	1636	1472	1960	2623	4734	3277	2925	1617
	100	10^8	8577	7831	10187	14217	24220	14096	13338	8801
	500	10^8	2274	2061	2682	3320	5002	3439	3497	2337
	1000	10^8	1404	1273	1702	2043	2992	2094	2093	1439
	5000	10^7	5816	5223	6769	7814	9925	8020	8006	–
	10000	10^7	4118	3670	4761	5384	6536	5513	5466	–
Higgs	50	10^5	1302	1244	1724	1844	1826	1869	1812	1268
	100	10^5	1199	1135	1512	1630	1610	1625	1634	1153
	500	10^4	9869	9148	11477	12340	12111	12127	12152	8822
	1000	10^4	9047	8363	10203	10912	10805	10773	10830	7942
	5000	10^4	7388	6887	7913	8417	8318	8411	8351	6338
	10000	10^4	6802	6339	7118	7560	7459	7584	7536	5745
Gaussian	50	10^0	206	206	315	341	2852589	2789288	4568897	10^{106}
	100	10^0	142	141	210	235	1305797	1265933	1197539	10^{108}
	500	10^0	126	120	151	166	77767	39332	21360	10^{104}
	1000	10^0	120	111	134	146	12391	21333	16572	$5 \cdot 10^8$
	5000	10^0	104	95	102	111	4886	165	165	$5 \cdot 10^8$
	10000	10^0	97	89	92	99	153	154	153	10^{107}
High	50	10^2	7983	7982	13684	22250	39381	33426	38672	8873
	100	10^0	51	51	100	103	2195989	1653400	2245462	51
	500	10^0	50	50	93	95	177184	95271	69258	–
Image	50	10^4	3321	2861	3322	4254	7459	4680	4797	2897
	100	10^4	1738	1489	1693	2217	4448	2314	2191	1505
	500	10^3	3589	3181	3541	4218	12389	4530	4397	3258
Mnist	50	10^8	1524	1432	2273	2451	2418	2409	2435	1431
	100	10^8	1398	1328	2017	2162	2126	2143	2161	1309
	500	10^8	1149	1085	1561	1672	1651	1666	1660	–

Table 3: Memory usage (in megabytes) by algorithm, dataset (D), and cluster count k. We abbreviated *SOCCER* as *SOCC* and *k-Means++* as *kM++*.

D	k	**BSP**	Local	kM++	Rand	Fast	Reject	SOCC
Protein	50	257	184	177	176	11882	11931	608
	100	257	184	177	177	11881	11931	816
	500	258	184	177	177	11883	11931	1032
	1000	258	185	178	177	11901	11931	–
	5000	262	187	180	180	11885	11934	–
	10000	266	190	183	183	11896	11962	–
Song	50	533	397	385	383	27623	27761	1114
	100	534	398	385	383	27623	27760	1499
	500	534	398	385	383	27626	27760	1683
	1000	534	398	386	384	27674	27752	2023
	5000	538	401	389	387	27627	27765	–
	10000	543	405	392	390	27633	27758	–
Census	50	1983	1459	1400	1382	21989	22231	3489
	100	1983	1459	1400	1382	21990	22231	3868
	500	1984	1459	1400	1382	21989	22232	4088
	1000	1984	1459	1400	1383	21980	22231	4310
	5000	1987	1462	1403	1385	21993	22233	6263
	10000	1991	1465	1405	1388	21994	22236	–
Covertype	50	410	278	265	263	15538	15516	807
	100	410	278	265	263	15528	15516	1077
	500	411	278	265	263	15528	15508	1184
	1000	411	279	266	263	15529	15516	1432
	5000	414	281	267	265	15535	15510	–
	10000	417	283	270	268	15559	15521	–
Higgs	50	5829	2998	2734	2648	42108	43030	5693
	100	5828	2998	2734	2648	42094	43108	5766
	500	5832	2998	2734	2648	42131	43058	5712
	1000	5831	2998	2734	2648	42198	43051	5755
	5000	5831	2999	2735	2649	42241	43136	6478
	10000	5833	3001	2736	2651	42197	43050	7609
Gaussian	50	4318	1684	1444	1366	6650	6650	2918
	100	4319	1684	1444	1366	6652	6650	2976
	500	4317	1684	1444	1366	6652	6650	2751
	1000	4321	1684	1444	1366	6650	6651	2996
	5000	4319	1684	1444	1366	6651	6650	3485
	10000	4321	1685	1445	1367	6652	6651	4125
High	50	442	416	415	415	1299	1299	1079
	100	443	416	415	415	1299	1299	1321
	500	444	418	416	417	1301	1301	–
Image	50	64	35	22	19	316	312	192
	100	64	35	22	19	316	313	237
	500	65	35	22	20	315	312	277
Mnist	50	463	445	443	442	21691	21695	1405
	100	463	445	443	443	21687	21689	1645
	500	466	448	446	445	21692	21689	–

Table 4: Average and maximum BSP tree depth by dataset (D). A full BSP tree was created, subdividing nodes until all nodes have a variance of zero. Additionally, the logarithm of the dataset size is given.

D	Avg Depth	Max Depth	$\ln(n)$
Protein	20.4	30	18.1
Song	20.6	29	19.0
Census	25.6	42	21.2
Covertype	20.3	28	19.0
Higgs	24.5	34	23.4
Gaussian	24.4	33	23.3
High	21.0	46	16.6
Image	17.6	23	19.0
Mnist	17.1	25	16.1

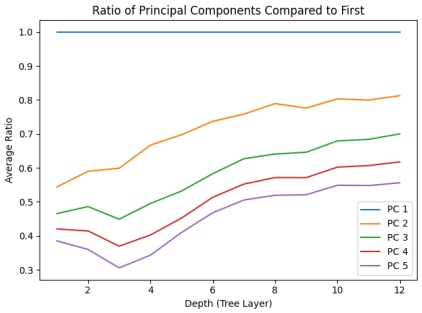


Figure 2: Ratios of the ith principal component and the first across tree depths. The ratios are averaged across all datasets (excluding *Image* and *Mnist*, see Section 4.4) and nodes in a layer. We observe increasing isotropy in deeper layers.

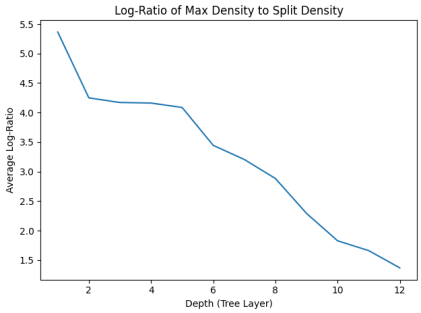


Figure 3: Average log-ratio between maximum density and the density at the split index, across tree depths. We observe that the algorithm prefers splitting in regions of low density alongside a homogenization of density in deeper layers.

References

[1] Marcel R. Ackermann, Marcus Märtens, Christoph Raupach, Kamil Swierkot, Christiane Lammersen, and Christian Sohler. 2012. StreamKM++: a clustering algorithm for data streams. *ACM Journal of Experimental Algorithmics* 17, 2, Article 2.4 (2012), 30 pages. doi:10.1145/2133803.2184450

[2] Sara Ahmadian, Ashkan Norouzi-Fard, Ola Svensson, and Justin Ward. 2019. Better guarantees for k-means and Euclidean k-median by primal-dual algorithms. *SIAM Journal on Computing* 49, 4 (2019), FOCS17–97. doi:10.1137/18M1171321

[3] Daniel Aloise, Amit Deshpande, Pierre Hansen, and Preyas Popat. 2009. NP-hardness of Euclidean sum-of-squares clustering. *Machine Learning* 75 (2009), 245–248. doi:10.1007/s10994-009-5103-0

[4] David Arthur and Sergei Vassilvitskii. 2007. k-means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '07)*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1027–1035. https://dl.acm.org/doi/10.5555/1283383.1283494

[5] Olivier Bachem, Mario Lucic, Hamed Hassani, and Andreas Krause. 2016. Fast and provably good seedings for k-means. In *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (Eds.), Vol. 29. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2016/file/d67d8ab4f4c10bf22aa353e27879133c-Paper.pdf

[6] Mark Bagnoli and Ted Bergstrom. 2005. Log-concave probability and its applications. *Economic Theory* 26, 2 (2005), 445–469. doi:10.1007/s00199-004-0514-4

[7] Bahman Bahmani, Benjamin Moseley, Andrea Vattani, Ravi Kumar, and Sergei Vassilvitskii. 2012. Scalable k-means++. *Proceedings of the VLDB Endowment* 5, 7 (2012), 622–633. doi:10.14778/2180912.2180915

[8] Lorenzo Beretta, Vincent Cohen-Addad, Silvio Lattanzi, and Nikos Parotsidis. 2023. Multi-swap k-means++. *Advances in Neural Information Processing Systems* 36 (2023), 26069–26091. https://papers.nips.cc/paper_files/paper/2023/file/52d63f9e4b81f866bf69fb3c834aad47-Paper-Conference.pdf

[9] T. Bertin-Mahieux. 2011. Year Prediction MSD. UCI Machine Learning Repository. doi:10.24432/C50K61

[10] Jock Blackard. 1998. Covertype. UCI Machine Learning Repository. doi:10.24432/C50K5N

[11] Stefano Bruno, Ying Zhang, Dong-Young Lim, Ömer Deniz Akyildiz, and Sotirios Sabanis. 2025. On diffusion-based generative models and their error bounds: The log-concave case with full convergence estimates. arXiv:2311.13584 [cs.LG] doi:10.48550/arXiv.2311.13584

[12] CCCL Development Team. 2023. *CCCL: CUDA C++ Core Libraries*. https://github.com/NVIDIA/cccl

[13] Davin Choo, Christoph Grunau, Julian Portmann, and Vaclav Rozhon. 2020. k-means++: few more steps yield constant approximation. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*, Hal Daumé III and Aarti Singh (Eds.). PMLR, 1909–1917. https://proceedings.mlr.press/v119/choo20a.html

[14] Vincent Cohen-Addad, Hossein Esfandiari, Vahab Mirrokni, and Shyam Narayanan. 2022. Improved approximations for Euclidean k-means and k-median, via nested quasi-independent sets. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022)*. Association for Computing Machinery, New York, NY, USA, 1621–1628. doi:10.1145/3519935.3520011

[15] Vincent Cohen-Addad, Silvio Lattanzi, Ashkan Norouzi-Fard, Christian Sohler, and Ola Svensson. 2020. Fast and accurate k-means++ via rejection sampling. *Advances in Neural Information Processing Systems* 33 (2020), 16235–16245. https://papers.nips.cc/paper/2020/file/babcff88f8be8c4795bd6f0f8cccca61-Paper.pdf

[16] Sanjoy Dasgupta. 2008. *The hardness of k-means clustering*. Technical Report CS2008-0916. Department of Computer Science and Engineering, University of California, San Diego, CA, USA. https://escholarship.org/uc/item/2qm3k10c

[17] Fabrizio Grandoni, Rafail Ostrovsky, Yuval Rabani, Leonard J. Schulman, and Rakesh Venkat. 2022. A refined approximation for Euclidean k-means. *Information Processing Letters* 176 (2022), 106251. doi:10.1016/j.ipl.2022.106251

[18] Tom Hess, Ron Visbord, and Sivan Sabato. 2023. Fast distributed k-means with a small number of rounds. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 206)*, Francisco Ruiz, Jennifer Dy, and Jan-Willem van de Meent (Eds.). PMLR, 850–874. https://proceedings.mlr.press/v206/hess23a.html

[19] Adam R. Klivans, Philip M. Long, and Alex K. Tang. 2009. Baum's algorithm learns intersections of halfspaces with respect to log-concave distributions. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*, Irit Dinur, Klaus Jansen, Joseph Naor, and José Rolim (Eds.). Springer, Berlin, Heidelberg, Germany, 588–600. doi:10.1007/978-3-642-03685-9_44

[20] Nojun Kwak. 2008. Principal component analysis based on L1-norm maximization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30, 9 (2008), 1672–1680. doi:10.1109/TPAMI.2008.114

[21] Sebastian Künzel and Daniel Weiskopf. 2025. Replication Data for: "BSP k-Means". DaRUS. doi:10.18419/DARUS-5575

[22] Silvio Lattanzi and Christian Sohler. 2019. A better k-means++ algorithm via local search. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 3662–3671. https://proceedings.mlr.press/v97/lattanzi19a.html

[23] Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. [n. d.]. MNIST Database of Handwritten Digits. https://openml.org/search?type=data&id=554 Accessed: 2025-11-28.

[24] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137. doi:10.1109/TIT.1982.1056489

[25] László Lovász and Santosh Vempala. 2007. The geometry of logconcave functions and sampling algorithms. *Random Structures & Algorithms* 30, 3 (2007), 307–358. doi:10.1002/rsa.20135

[26] Meena Mahajan, Prajakta Nimbhorkar, and Kasturi Varadarajan. 2012. The planar k-means problem is NP-hard. *Theoretical Computer Science* 442 (2012), 13–21. doi:10.1016/j.tcs.2010.05.034

[27] Pascal Massart. 1990. The tight constant in the Dvoretzky-Kiefer-Wolfowitz inequality. *The Annals of Probability* 18, 3 (1990), 1269–1283. doi:10.1214/aop/1176990746

[28] Chris Meek, Bo Thiesson, and David Heckerman. 2001. US Census Data (1990). UCI Machine Learning Repository. doi:10.24432/C5VP42

[29] Nobuyuki Otsu. 1979. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics* 9, 1 (1979), 62–66. doi:10.1109/TSMC.1979.4310076

[30] SIGKDD Web Page. 2004. KDD Cup 2004: Particle physics; plus protein homology prediction. https://www.kdd.org/kdd-cup/view/kdd-cup-2004/Data Accessed: 2025-02-10.

[31] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830. http://jmlr.org/papers/v12/pedregosa11a.html

[32] David Pollard. 1981. Strong consistency of k-means clustering. *The Annals of Statistics* 9, 1 (1981), 135–140. doi:10.1214/aos/1176345339

[33] Adrien Saumard and Jon A. Wellner. 2014. Log-concavity and strong log-concavity: a review. *Statistics Surveys* 8 (2014), 45–114. doi:10.1214/14-SS107

[34] Chris Schwiegelshohn and Omar Ali Sheikh-Omar. 2022. An empirical evaluation of k-means coresets. In *European Symposium on Algorithms (Leibniz International Proceedings in Informatics (LIPIcs))*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 84:1–84:17. doi:10.4230/LIPIcs.ESA.2022.84

[35] Pooya Tavallali, Peyman Tavallali, and Mukesh Singhal. 2021. K-means tree: an optimal clustering tree for unsupervised learning. *The Journal of Supercomputing* 77, 5 (2021), 5239–5266. doi:10.1007/s11227-020-03436-2

[36] Robert L. Thorndike. 1953. Who belongs in the family? *Psychometrika* 18, 4 (1953), 267–276. doi:10.1007/BF02289263

[37] Daniel Whiteson. 2014. HIGGS. UCI Machine Learning Repository. doi:10.24432/C5V312

A Logarithmic Tree Depth under Recursive Otsu Splitting of Log-Concave Densities

Goal. Assuming a log-concave input distribution, show that the depth D of the deepest root-to-leaf path in the sample-labeled population Otsu tree, given n datapoints, satisfies:

$$D \in O(\ln n) \quad \text{with probability } 1 - \delta,$$

for any fixed confidence parameter value $\delta \in (0, 1)$.

Limitations of this Analysis. *BSP k-Means* constructs an *empirical* Otsu tree, choosing at each node the threshold that maximizes empirical between-class variance (BCV). In this section, we instead analyze an idealized *sample-labeled population Otsu tree*, obtained by splitting at the population BCV-maximizing thresholds and then routing the n samples through this fixed tree. This object represents the large-sample limit of the empirical procedure. Bridging the gap between the empirical and population trees requires additional arguments that we defer to future work due to space constraints, in particular: (i) a uniform high-probability bound on the deviation between empirical and population BCV as a function of the threshold, and (ii) a stability result showing that any empirical BCV maximizer inherits the population-level balance property of Lemma A.2.

A.1 Preservation of Log-Concavity

Every operation in the recursive procedure preserves log-concavity:

- **Re-centering** $x \mapsto x - \mu$: Affine transformations preserve log-concavity (see Saumard and Wellner, Section 3.1 [33]).
- **Projection to 1D:** The Prékopa–Leindler theorem implies that every marginal of a log-concave distribution is log-concave (see Saumard and Wellner, Section 3.1 [33]).
- **Half-space cut:** The restriction of a log-concave distribution to a truncated subset remains log-concave (see Bagnoli and Bergstrom, Theorem 9 [6]).

Hence, at each node, the projected 1D distribution given to Otsu's method is log-concave.

A.2 Depth Guarantee via DKW Inequality

Fix a log-concave distribution P on $\mathbb{R}^d$. At each node v with region A_v, let $P_v := \mathcal{L}(X \mid X \in A_v)$ be the conditional distribution of P restricted to A_v, and let F_v be the CDF of the 1D projection used at v. By Appendix A.1, the projected density f_v is log-concave. Let $t_v^\star$ be any population BCV-maximizing threshold for F_v as in Lemma A.2, and split A_v at $t_v^\star$. Thus the collection of regions $\{A_v\}$ and thresholds $\{t_v^\star\}$ is determined solely by P.

Now draw n i.i.d. samples $X_1, \ldots, X_n \sim P$ and route them through these fixed splits. For each node v, let M_v be the number of sample points in A_v, and let $F_{\mathrm{emp},v}$ be the empirical CDF of the projected values in that node. Conditional on $M_v = m$, the m points in A_v are i.i.d. from P_v, so $F_{\mathrm{emp},v}$ is the empirical CDF of an i.i.d. sample from F_v, and the Dvoretzky–Kiefer–Wolfowitz inequality applies [27].

Let $\delta \in (0,1)$ be a confidence parameter. For a node v with $M_v = m$ and population CDF F_v, the two-sided DKW inequality yields, for any $\delta_{\mathrm{node}} \in (0,1)$,

$$\Pr\Big(\sup_x |F_{\mathrm{emp},v}(x) - F_v(x)| > \varepsilon \,\Big|\, M_v = m\Big) \;\le\; 2e^{-2m\varepsilon^2}.$$

So, with probability at least $1 - \delta_{\mathrm{node}}$,

$$|F_{\mathrm{emp},v}(x) - F_v(x)| \;\le\; \varepsilon(m, \delta_{\mathrm{node}}) := \sqrt{\frac{\ln(2/\delta_{\mathrm{node}})}{2m}} \quad \text{for all } x.$$

Balance from Log-Concavity. By Lemma A.2, there exists a universal constant $c > 0$ such that, for every node v and its population Otsu threshold $t_v^\star$,

$$w_{v,0}(t_v^\star) = F_v(t_v^\star) \;\ge\; c, \qquad w_{v,1}(t_v^\star) = 1 - F_v(t_v^\star) \;\ge\; c.$$

Evaluating the DKW bound at $x = t_v^\star$ gives, on the DKW event,

$$w_{v,0}^{(\mathrm{emp})}(t_v^\star) = F_{\mathrm{emp},v}(t_v^\star) \;\ge\; F_v(t_v^\star) - \varepsilon \;\ge\; c - \varepsilon,$$

and similarly for $w_{v,1}^{(\mathrm{emp})}(t_v^\star)$. Thus, if we choose $\varepsilon \le c/2$, both empirical class weights at $t_v^\star$ satisfy

$$w_{v,i}^{(\mathrm{emp})}(t_v^\star) \;\ge\; \frac{c}{2},$$

so empirical splits at the population thresholds are balanced.

Node-wise Balance Guarantee. To enforce $\varepsilon \le c/2$, we require

$$\varepsilon(m, \delta_{\mathrm{node}}) = \sqrt{\frac{\ln(2/\delta_{\mathrm{node}})}{2m}} \;\le\; \frac{c}{2} \quad\Rightarrow\quad m \;\ge\; \frac{2}{c^2}\ln\!\left(\frac{2}{\delta_{\mathrm{node}}}\right).$$

A full binary tree with at most n leaves has at most n internal nodes and at most $2n$ total nodes. Setting $\delta_{\mathrm{node}} = \delta/(2n)$ and applying a union bound over these nodes ensures that, with probability at least $1 - \delta$, every node v with

$$M_v \;\ge\; m_{\min}(n, \delta) := \frac{2}{c^2}\ln\!\left(\frac{4n}{\delta}\right)$$

simultaneously satisfies the $\frac{c}{2} : 1 - \frac{c}{2}$ empirical balance condition at $t_v^\star$.

Depth Bound. Each such balanced split reduces the number of points in the larger child by a factor of at most $1 - \frac{c}{2}$. Along any root-to-leaf path that continues splitting while $M_v \ge m_{\min}(n, \delta)$, the node sizes therefore satisfy

$$m_{\min}(n, \delta) \;\le\; \big(1 - \tfrac{c}{2}\big)^k n \quad\Rightarrow\quad k \;\le\; \frac{\ln\big(n/m_{\min}(n,\delta)\big)}{\ln\big(1/(1-\frac{c}{2})\big)}.$$

Once the node size drops below $m_{\min}(n, \delta)$, we no longer have a high-probability balance guarantee; in the worst case, we may remove a single point per split, requiring at most $m_{\min}(n, \delta)$ additional levels.

High-probability Bound. For every confidence level $\delta \in (0,1)$, with probability at least $1 - \delta$, the depth D of the sample-labeled population Otsu tree satisfies

$$D \;\le\; \frac{\ln\big(n/m_{\min}(n,\delta)\big)}{\ln\big(1/(1-\frac{c}{2})\big)} \;+\; m_{\min}(n,\delta),\; m_{\min}(n,\delta) = \frac{2}{c^2}\ln\!\left(\frac{4n}{\delta}\right).$$

In particular, for fixed δ, we have $D \in O(\ln n)$.

A.3 Minimum Class Weight Guarantee for Log-Concave Distributions

We apply Otsu's method to a projected, normalized, and re-centered 1D distribution. Due to re-centering by our algorithm and the preservation properties shown earlier, each such distribution is log-concave, and has zero mean. Additionally, we can assume any scaling since the optimal class weights of Otsu's method are invariant under a constant scale factor. We choose to scale to unit variance.

Between-Class Variance Sandwich for Log-Concave Distributions. Recall that for a threshold $t \in \mathbb{R}$ we write

$$w_0(t) = \Pr(X < t), \qquad w_1(t) = \Pr(X \ge t),$$
$$\mu_0(t) := \mathbb{E}[X \mid X < t], \qquad \mu_1(t) := \mathbb{E}[X \mid X \ge t],$$
$$w(t) := \min\{w_0(t), w_1(t)\}, \qquad \Delta(t) := \mu_1(t) - \mu_0(t).$$

The between-class variance (BCV) at threshold t is

$$\sigma_b^2(t) \;=\; w_0(t)\, w_1(t)\, \Delta(t)^2 \;=\; w(t)\big(1 - w(t)\big)\, \Delta(t)^2.$$

Lower Bound on BCV at a Central Threshold. Let $t_0 = 0$ be the mean (after centering). By Lemma A.3, there is a universal $c_1 > 0$ such that

$$w_0(t_0),\, w_1(t_0) \;\ge\; c_1.$$

By Lemma A.4, there exists a universal constant $c_2 > 0$ such that the right partial mean $\mu^+ := \mathbb{E}[X \mid X \ge 0]$ satisfies $\mu^+ \ge c_2$. Since $\mathbb{E}X = 0$ and $\Pr(X \ge 0), \Pr(X < 0) \ge c_1$, the left partial mean $\mu^- := \mathbb{E}[X \mid X < 0]$ is negative and bounded away from 0 in absolute value by another universal constant. Thus there exists $c_2' > 0$ such that

$$\Delta(t_0) \;=\; \mu^+ - \mu^- \;\ge\; c_2'.$$

Hence

$$\sigma_b^2(t_0) \;=\; w_0(t_0)\, w_1(t_0)\, \Delta(t_0)^2 \;\geq\; c_1^2 (c_2')^2 \;=:\; \alpha \;>\; 0,$$

where α is a universal constant depending only on the log-concave class.

Lemma A.1 (Upper Bound on BCV for Small Class Weight). *Let X be centered, unit-variance, and log-concave on $\mathbb{R}$. Then, there exists a universal constant $C > 0$ such that, whenever $w(t) \leq 1/2$,*

$$\sigma_b^2(t) \;\leq\; C\, w(t) \left(\ln(1/w(t))\right)^2.$$

In particular, $\sigma_b^2(t) \to 0$ as $w(t) \to 0$, uniformly over all such X.

Proof. By symmetry between the two classes, it suffices to treat the case where $t \geq 0$ and $w(t) = w_1(t) = \Pr(X \geq t)$; if instead $w(t) = w_0(t)$ is smaller, apply the same argument to $-X$.

Tail Bound and Relation Between t and $w(t)$. By Lemma 5.7 of Lovász and Vempala [25], any 1D log-concave X with $\mathbb{E}X^2 \leq 1$ satisfies

$$\Pr(|X| > u) \;<\; e^{-u+1} \quad \text{for all } u > 1.$$

In our setting, X is already centered and unit-variance, so this applies directly. For $u > 1$,

$$\Pr(X \geq u) \;\leq\; \Pr(|X| > u) \;\leq\; e^{-u+1} \;=\; e\, e^{-u},$$

and for $0 \leq u \leq 1$, we trivially have $\Pr(X \geq u) \leq 1 \leq e\, e^{-u}$. Thus, for all $u \geq 0$,

$$\Pr(X \geq u) \;\leq\; e\, e^{-u}.$$

In particular, for our threshold t,

$$w(t) = \Pr(X \geq t) \;\leq\; e\, e^{-t} \quad\Rightarrow\quad \frac{w(t)}{e} \;\leq\; e^{-t}$$

$$\Rightarrow\quad t \;\leq\; \ln \frac{e}{w(t)} = 1 + \ln \frac{1}{w(t)}.$$

So we have the upper bound $t \;\leq\; 1 + \ln \frac{1}{w(t)}$.

Bounding the Mean Overshoot and the Mean Gap. We next control $\mu_1(t) = \mathbb{E}[X \mid X \geq t]$. Consider the conditional random variable $Y := X \mid X \geq 0$. This is a truncation of a log-concave distribution, so Y also has a log-concave density by Theorem 9 of Bagnoli and Bergstrom [6]. For a nonnegative random variable with log-concave density, Bagnoli and Bergstrom show that the failure rate is increasing and that the mean residual lifetime

$$\mathrm{MRL}(x) := \mathbb{E}[Y - x \mid Y \geq x]$$

is decreasing in x (Theorem 6 and Corollary 4 of [6]). Applying this to Y gives, for all $t \geq 0$,

$$\mathbb{E}[X - t \mid X \geq t] = \mathbb{E}[Y - t \mid Y \geq t] \leq \mathbb{E}[Y \mid Y \geq 0] = \mathbb{E}[X \mid X \geq 0].$$

By Lemma A.3, $\Pr(X \geq 0) \geq c_1 > 0$, and Lemma 5.7 of Lovász and Vempala [25] also implies that $\mathbb{E}|X| \leq 2$ via tail integration. Thus,

$$\mathbb{E}[X \mid X \geq 0] \;\leq\; \frac{\mathbb{E}|X|}{\Pr(X \geq 0)} \;\leq\; \frac{2}{c_1} \;=:\; B,$$

so for every $t \geq 0$,

$$\mu_1(t) = \mathbb{E}[X \mid X \geq t] \;\leq\; t + B.$$

For the left class mean, we have the crude bound

$$|\mu_0(t)| \;\leq\; \mathbb{E}|X| \;\leq\; 2,$$

so for some universal constant $C_0 > 0$,

$$|\mu_0(t)| \leq C_0,\; \Delta(t) = \mu_1(t) - \mu_0(t) \;\leq\; |\mu_1(t)| + |\mu_0(t)| \;\leq\; t + B + C_0.$$

Combining this with the upper bound on t in terms of $w(t)$, we have

$$\Delta(t) \;\leq\; t + B + C_0 \;\leq\; \ln \frac{1}{w(t)} + (B + C_0 + 1) \;\leq\; C \ln \frac{1}{w(t)},$$

for some universal constant $C > 0$ (using $w(t) \leq 1/2$ so that $\ln(1/w(t)) \geq \ln 2 > 0$).

Bounding BCV. Finally,

$$\sigma_b^2(t) = w(t)\big(1 - w(t)\big)\Delta(t)^2 \;\leq\; w(t)\, \Delta(t)^2 \;\leq\; w(t)\, C^2 \big(\ln(1/w(t))\big)^2.$$

Thus, the claimed inequality holds with C replaced by C^2, and since $w \ln^2(1/w) \to 0$ as $w \to 0$, this upper bound tends to zero uniformly over all centered, unit-variance log-concave X. □

Excluding Extremely Unbalanced Splits. We can now combine the lower and upper bounds to obtain a universal balance constant for BCV-maximizing thresholds.

Lemma A.2 (Population-Level Balance of Otsu Thresholds). *Let X be centered, unit-variance, log-concave on $\mathbb{R}$, with CDF F, and let $t^\star \in \arg\max_t \sigma_b^2(t)$ be any population BCV-maximizing threshold. Then there exists a universal constant $c \in (0, 1/2)$ such that*

$$\min\{w_0(t^\star), w_1(t^\star)\} \;\geq\; c.$$

Proof. From the central-threshold argument above, we have a universal $\alpha > 0$ such that $\sigma_b^2(t_0) \geq \alpha$. By Lemma A.1, there exists a $c \in (0, 1/2)$ such that

$$C_1\, w \left(\ln(C_2/w)\right)^2 \;<\; \alpha \quad \text{for all } 0 < w \leq c.$$

For any threshold t with $w(t) \leq c$, the lemma gives $\sigma_b^2(t) < \alpha \leq \sigma_b^2(t_0)$, so such thresholds cannot maximize the BCV. Therefore, any BCV-maximizing threshold $t^\star$ must satisfy $w(t^\star) \geq c$, i.e.,

$$\min\{w_0(t^\star), w_1(t^\star)\} \;\geq\; c.$$

□

A.4 Lower Bound on Partial Mean Separation under Log-Concavity

Lemma A.3 (Balanced Mass Condition). *Let X be a centered, log-concave random variable on $\mathbb{R}$. Then,*

$$\Pr(X \leq 0) \geq \frac{1}{e}, \qquad \Pr(X \geq 0) \geq \frac{1}{e}.$$

This follows immediately from Klivans et al. [19], Lemma 5. The result holds without assuming unit variance, as positive scaling does not affect the probability bounds.

Lemma A.4 (Lower Bound on Right Partial Mean for Log–Concave Distributions). *Let X be a centered, unit-variance, log-concave random variable on $\mathbb{R}$. Define*

$$\mu^+ := \mathbb{E}[X \mid X \geq 0].$$

Then there exists a universal constant $c_2 > 0$ such that $\mu^+ \geq c_2$.

Proof. Write $P_+ := \Pr(X \geq 0)$ and $P_- := \Pr(X < 0)$, and

$$\mu^+ := \mathbb{E}[X \mid X \geq 0], \qquad \mu^- := \mathbb{E}[X \mid X < 0].$$

Note that $\mu^- < 0$.

Step 1: A Universal Lower Bound on $\mathbb{E}|X|$. By Theorem 5.22 of Lovász and Vempala [25], for any log-concave random vector Y in $\mathbb{R}^n$ and any $k \geq 1$,

$$\left(\mathbb{E}|Y|^k\right)^{1/k} \;\leq\; 2k\,\mathbb{E}|Y|.$$

Apply this with $n = 1$, $k = 2$, and $Y = X$. Since X is centered and has unit variance, we have $\mathbb{E}X^2 = 1$, hence

$$1^{1/2} = \left(\mathbb{E}X^2\right)^{1/2} \;\leq\; 4\,\mathbb{E}|X|,$$

so

$$\mathbb{E}|X| \;\geq\; \frac{1}{4}.$$

Step 2: Expressing $\mathbb{E}|X|$ *in Terms of* μ^+. Using the decomposition by sign,

$$\mathbb{E}|X| = \mathbb{E}[|X| \mid X \geq 0]P_+ + \mathbb{E}[|X| \mid X < 0]P_- = \mu^+ P_+ + (-\mu^-)P_-.$$

Since $\mathbb{E}X = 0$, we also have

$$0 = \mathbb{E}X = \mu^+ P_+ + \mu^- P_-,$$

so $\mu^- P_- = -\mu^+ P_+$. Plugging this into the expression for $\mathbb{E}|X|$ gives

$$\mathbb{E}|X| = \mu^+ P_+ - \mu^- P_- = \mu^+ P_+ - (-\mu^+ P_+) = 2\mu^+ P_+.$$

Hence,

$$\mu^+ = \frac{\mathbb{E}|X|}{2P_+}.$$

Step 3: Final Lower Bound. We have $P_+ \leq 1$, so

$$\mu^+ = \frac{\mathbb{E}|X|}{2P_+} \;\geq\; \frac{\mathbb{E}|X|}{2} \;\geq\; \frac{1/4}{2} = \frac{1}{8}.$$

Thus, the lemma holds with $c_2 = 1/8$, a universal constant. □

A.5 Extension to Mixtures of Log-Concave Distributions

We briefly outline how the depth bound extends to finite mixtures of log-concave distributions. Let

$$P = \sum_{k=1}^{K} \pi_k P_k$$

be a finite mixture of log-concave components on $\mathbb{R}^d$. The tree-generation operations used by our procedure—global affine re-centering and rescaling, 1D projection, and restriction to half-spaces—all preserve log-concavity componentwise. Thus, at any node v, the conditional distribution can be written as

$$P_v = \sum_{k=1}^{K_v} \pi_{v,k} P_{v,k},$$

where each $P_{v,k}$ is log-concave, obtained from some P_k by restriction and an affine transform. The mixture weights $\pi_{v,k}$ may change along the tree.

To control the effect of small-weight components far in the tails, we impose a global weight floor:

Assumption 1 (Global Mixture Weight Floor). *There exists* $\pi_{\min} > 0$ *such that, for every node* v *and every component* k *in the mixture representation of* P_v,

$$\pi_{v,k} \;\geq\; \pi_{\min}.$$

At each node v, consider the 1D projected distribution X_v used by Otsu's method. By affine invariance of the BCV objective, we may globally re-center and rescale X_v so that one component (e.g., any component with weight at least $\pi_{\min}$) becomes centered and unit-variance. For this component, all single-component log-concave arguments from Appendix A.3 and A.4 apply unchanged. The mixture weight floor $\pi_{\min}$ then allows us to transfer the resulting BCV lower and upper bounds to the full mixture X_v, up to constants depending only on $\pi_{\min}$ and the log-concave class.

Consequently, the BCV "sandwich" from Appendix A.3 holds for mixtures with modified constants: there exist $\alpha_{\text{mix}} > 0$, $C_{\text{mix}} > 0$, and $c_{\text{mix}} \in (0, 1/2)$ such that, at each node v, (i) the BCV at a central threshold is at least α_{mix}, and (ii) for any threshold with smaller class weight $w(t) \leq c_{\text{mix}}$,

$$\sigma_b^2(t) \;\leq\; C_{\text{mix}}\, w(t) \left(\ln(1/w(t))\right)^2.$$

Exactly as in Lemma A.2, this implies that any population BCV-maximizing threshold $t_v^\star$ at node v is c_{mix}-balanced:

$$\min\{w_{v,0}(t_v^\star), w_{v,1}(t_v^\star)\} \;\geq\; c_{\text{mix}}.$$

Replacing the balance constant c by c_{mix} in the high-probability analysis of Appendix A.2 yields the same logarithmic depth guarantee.

B Elbow Method Example

We provide an example application of *BSP k-Means* using the *High* dataset. *BSP k-Means* is used to compute clusters and clustering costs for $k \leq 200$. Since *High* is a mixture of 100 Gaussian distributions, we expect high cost for $k < 100$. As soon as each distribution in the mixture is represented by at least one sample, we expect the clustering cost to decrease marginally. The experiment confirms our assumption and shows a clear elbow in the log-cost plot (see Figure 4). We conclude that *BSP k-Means* correctly distributes samples for the given mixture and that the cost values are well-suited for the elbow method.

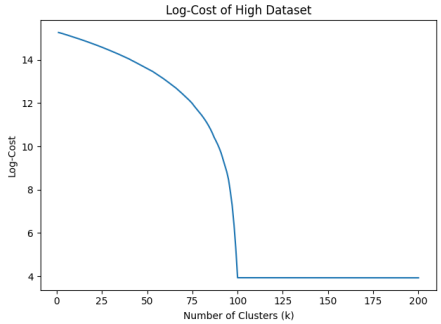


Figure 4: An example application of the elbow method and *BSP k-Means* using the *High* dataset. *BSP k-Means* is used to compute clusters and clustering costs for $k \leq 200$.